

DSA MINI PROJECT — CASE STUDY 5

Video Streaming Buffer Optimization

Determining Minimum Buffer Size for Stall-Free Playback

Language: C++ (Standard 17) | **DSA Concepts:** Binary Search, Prefix Sums, Sliding Window
Date: April 2026

Table of Contents

Problem Statement & Objective	2
The Core Idea — How Video Streaming Works	2
Data Structures & Algorithms Used	3
Algorithm 1 — Binary Search + Linear Simulation	4
Algorithm 2 — Closed-Form Optimal (Prefix Deficit)	6
Step-by-Step Worked Example	7
Additional Features — Network Analysis & Recommendations	9
Program Structure & Code Walkthrough	10

Sample Output	11
Test Results	12
Complexity Analysis	13
Conclusion	13

1. Problem Statement & Objective

 Objective

Determine the **minimum initial buffer size** (in KB) that ensures smooth, stall-free video playback given a sequence of fluctuating network bandwidth values.

What We're Given

- **Bandwidth array** `bandwidth[0..n-1]` — the network speed (KB/s) available at each second
- **Video bitrate** — the constant data consumption rate of the video player (KB/s)

What We Need to Find

The smallest value of `initialBuffer` such that the video plays from start to finish without ever stalling (buffer level never drops below zero).

Why This Matters

WITHOUT OPTIMIZATION	WITH OPTIMIZATION
Random buffer size → either too large (wastes memory/startup time) or too small (causes stalls)	Mathematically optimal buffer → zero stalls, minimum memory usage, instant playback
Poor user experience — buffering spinner	Smooth, uninterrupted viewing

2. The Core Idea — How Video Streaming Works

Think of the buffer as a **water tank**:

REAL WORLD	OUR MODEL
Water flows IN from a pipe	Network delivers <code>bandwidth[i]</code> KB each second

Water flows OUT through a tap	Video player consumes <code>bitrate</code> KB each second
Tank runs dry → no water	Buffer = 0 → video stalls (buffering spinner!)
Pre-fill the tank before opening the tap	Set an initial buffer before playback starts

Each second, the buffer changes by:

```
buffer = buffer + bandwidth[i] - bitrate
```

- If `bandwidth[i] > bitrate` → buffer **fills** (network is fast enough)
- If `bandwidth[i] < bitrate` → buffer **drains** (network is too slow)
- If `bandwidth[i] = bitrate` → buffer stays **steady**
- If buffer ever hits **0** → the video **stalls**

Key Insight

We want the **smallest** initial buffer that prevents the tank from ever running dry, no matter how the bandwidth fluctuates.

3. Data Structures & Algorithms Used

DSA CONCEPT	WHERE USED	PURPOSE
Binary Search	<code>findMinBufferBinarySearch()</code>	Search for the optimal buffer size in a sorted answer space
Linear Scan / Simulation	<code>canPlaySmoothly()</code>	Simulate playback second-by-second to check if a buffer works
Prefix Sums	<code>findMinBufferOptimal()</code>	Track running sum of net bandwidth to find maximum deficit
Sliding Window	<code>analyzeNetwork()</code>	Find the worst consecutive k-second bandwidth period
Greedy / Monotonicity	Binary Search logic	If buffer B works → B+1 also works (enables binary search)

Data Structures

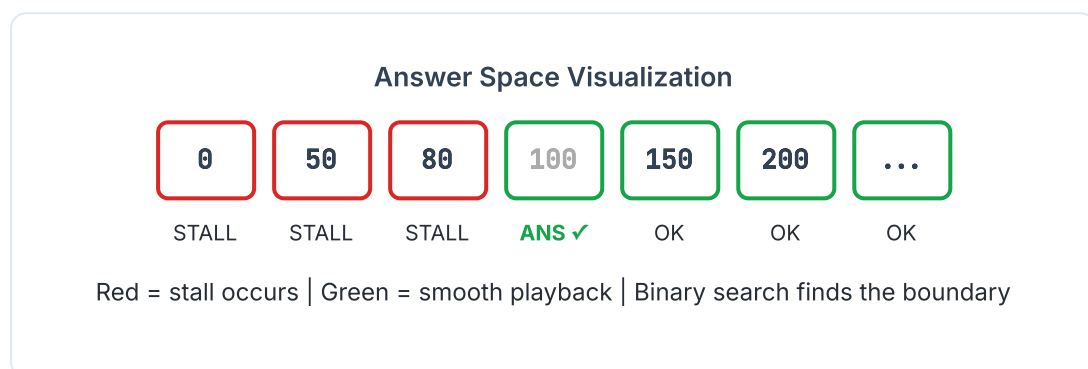
STRUCTURE	TYPE	PURPOSE
<code>vector<int> bandwidth</code>	Dynamic Array	Stores network speed for each second
<code>struct NetworkStats</code>	Struct	Groups all network analysis results (avg, jitter, worst window, etc.)
<code>struct TestCase</code>	Struct	Packages test inputs, expected output, and description

4. Algorithm 1 — Binary Search + Linear Simulation

This is the **primary approach** required by the case study.

The Monotonicity Property

The key observation that makes Binary Search possible: **If a buffer of size B allows smooth playback, then any buffer of size B+1 or larger also works.** This means the answer space is monotonic — all values below the answer fail, all values at or above the answer succeed.



How It Works — Step by Step

1

Set the search range

`lo = 0` (maybe no buffer needed) and `hi = n × bitrate` (worst case: zero bandwidth for n seconds).

2

Pick the midpoint

`mid = lo + (hi - lo) / 2` — overflow-safe calculation of the middle value.

3

Simulate playback with buffer = mid

Call `canPlaySmoothly(bandwidth, bitrate, mid)` — this runs a linear $O(n)$ simulation. Each second: `buffer += bandwidth[i] - bitrate`. If buffer ever goes below 0, return false.

4

Narrow the search

If smooth $\rightarrow$ this buffer works, but maybe we can do with less. Set `answer = mid`, then `hi = mid - 1`.

If stall $\rightarrow$ buffer too small. Set `lo = mid + 1`.

5

Repeat until `lo > hi`

The final value of `answer` is the minimum buffer that works.

The Code

```
bool canPlaySmoothly(const vector<int>& bandwidth, int bitrate, long long
    for (int bw : bandwidth) {
        buffer += bw - bitrate;
        if (buffer < 0) return false;    // stall!
    }
    return true;
}

long long findMinBufferBinarySearch(const vector<int>& bandwidth, int bitrate) {
    int n = bandwidth.size();
    long long lo = 0, hi = (long long)n * bitrate;
    long long answer = hi;

    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        if (canPlaySmoothly(bandwidth, bitrate, mid)) {
            answer = mid;        // works - try smaller
            hi = mid - 1;
        } else {
            lo = mid + 1;        // too small - try larger
        }
    }
    return answer;
}
```

5. Algorithm 2 — Closed-Form Optimal (Prefix Deficit)

This is a **bonus $O(n)$ approach** that computes the answer directly without any binary search.

The Mathematical Insight

Define `net[i] = bandwidth[i] - bitrate` — the gain or loss at each second.

The buffer level after k seconds is:

```
bufferLevel(k) = initialBuffer + net[0] + net[1] + ... + net[k-1]
                = initialBuffer + prefixSum(k)
```

For the buffer to **never go negative**:

```
initialBuffer + prefixSum(k) ≥ 0    for all k
initialBuffer ≥ -prefixSum(k)      for all k
initialBuffer ≥ -min(prefixSum)    // take the worst case
```

Formula

answer = max(0, -minPrefixSum)

Where minPrefixSum is the smallest running total of (bandwidth – bitrate) across all seconds.

The Code

```
long long findMinBufferOptimal(const vector<int>& bandwidth, int bitrate) {
    long long runningSum = 0;
    long long worstDeficit = 0;

    for (int bw : bandwidth) {
        runningSum += bw - bitrate;
        worstDeficit = min(worstDeficit, runningSum);
    }
}
```

```
    return max(0LL, -worstDeficit);  
}
```

Why This Works — Visual Explanation

Imagine plotting the running sum of net bandwidth over time. The deepest valley below zero tells you how much "pre-filling" the buffer needs:

Example: bandwidth = {50, 50, 200}, bitrate = 100

SECOND	BW	NET (BW-100)	RUNNING SUM	BUFFER IF START=0
1	50	-50	-50	-50 ← deficit!
2	50	-50	-100 ← worst!	-100 ← STALL
3	200	+100	0	0

Worst prefix sum = -100 → answer = $\max(0, -(-100))$ = **100 KB**

6. Step-by-Step Worked Example

Problem
Bandwidth: [120, 80, 60, 150, 40, 200, 30, 90] KB/s
Bitrate: 100 KB/s
Find: Minimum initial buffer for stall-free playback

Step 1: Compute Net Values and Running Sum

SECOND	BANDWIDTH	BITRATE	NET (BW – BITRATE)	RUNNING SUM
1	120	100	+20	+20
2	80	100	–20	0
3	60	100	–40	–40
4	150	100	+50	+10
5	40	100	–60	–50 ← worst deficit
6	200	100	+100	+50
7	30	100	–70	–20
8	90	100	–10	–30

Step 2: Apply the Formula

Minimum prefix sum = **–50** (at second 5)

`answer = max(0, –(–50)) = 50 KB`

Step 3: Verify with Simulation (buffer = 50)

SECOND	BW	NET	BUFFER LEVEL	STATUS
--------	----	-----	--------------	--------

1	120	+20	70	✅ filling
2	80	−20	50	✅ draining
3	60	−40	10	✅ draining
4	150	+50	60	✅ filling
5	40	−60	0	⚠️ just barely survived!
6	200	+100	100	✅ filling
7	30	−70	30	✅ draining
8	90	−10	20	✅ draining

Result

Buffer = 50 KB → **zero stalls**. The buffer touches exactly 0 at second 5 (the worst moment), confirming this is the minimum. Any smaller buffer (e.g., 49 KB) would stall at second 5.

Step 4: Binary Search also finds 50

Binary search range: [0, 800]. It takes ~10 steps to converge to 50 KB — same answer, just slower.

7. Additional Features — Network Analysis & Recommendations

Beyond finding the buffer, the program analyzes **why** a certain buffer is needed by examining network conditions.

7.1 Network Statistics

Metric	What it tells us	Algorithm
Average Bandwidth	Overall throughput — is the network fast enough on average?	Sum all bandwidth values, divide by n. $O(n)$
Min / Max	Range of fluctuation — how wild are the swings?	Track min/max in single pass. $O(n)$
Jitter (Std Dev)	How unstable the connection is. High jitter = unpredictable.	Compute variance, take square root. $O(n)$
Worst k-sec Window	The hardest consecutive period for the buffer to survive.	Sliding Window — classic DSA technique. $O(n)$
Stability Index	0–100 score combining headroom and consistency.	Weighted combination of headroom + jitter ratio. $O(1)$

7.2 Sliding Window — Finding the Worst Period

The **sliding window** technique efficiently finds the k consecutive seconds with the lowest total bandwidth:

```
// Calculate first window sum
int windowSum = 0;
for (int i = 0; i < windowSize; i++)
    windowSum += bandwidth[i];

// Slide the window: add new element, remove old one
for (int i = windowSize; i < n; i++) {
    windowSum += bandwidth[i] - bandwidth[i - windowSize];
    if (windowSum < worstWindowSum) {
        worstWindowSum = windowSum;
        worstWindowStart = i - windowSize + 1;
    }
}
```

```
}  
}
```

Instead of checking every possible window in $O(n \times k)$, the sliding window does it in **$O(n)$** by reusing the previous sum.

7.3 Buffer Recommendations

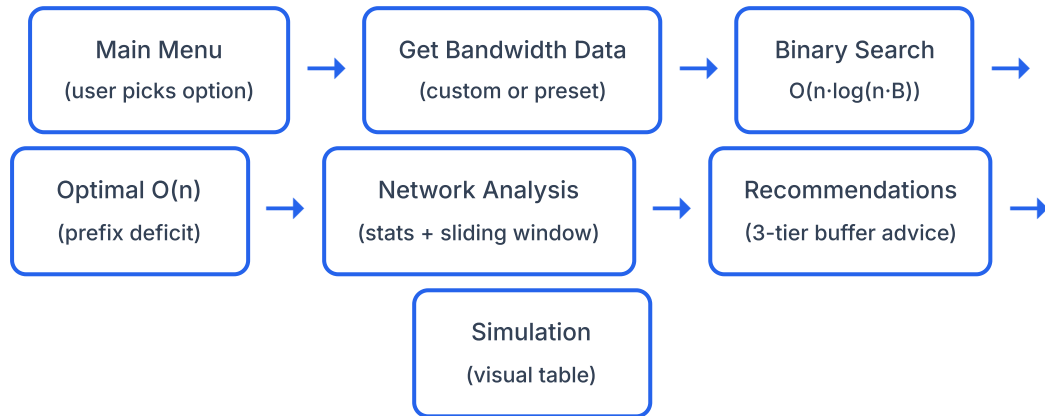
The program outputs three tiers of buffer recommendations:

TIER	MARGIN	USE CASE
MINIMUM	0%	Mathematically exact — any unexpected dip causes stall
SAFE	$+20\% \times (1 + \text{jitter ratio})$	Handles minor bandwidth fluctuations
ROBUST	$+50\% \times (1 + \text{jitter ratio})$	Handles major jitter and network spikes

The safety margin **scales dynamically with jitter** — an unstable network gets a proportionally larger margin than a stable one.

8. Program Structure & Code Walkthrough

Overall Flow



Functions Map

FUNCTION	LINES	PURPOSE
<code>printLine()</code>	~3	Utility — print a separator line
<code>readPositiveInt()</code>	~15	Input validation with retry and EOF handling
<code>canPlaySmoothly()</code>	~6	Core — simulate playback, return true/false
<code>findMinBufferBinarySearch()</code>	~25	Core — binary search on buffer size
<code>findMinBufferOptimal()</code>	~10	Core — $O(n)$ closed-form solution
<code>simulatePlayback()</code>	~35	Visual — second-by-second playback table
<code>analyzeNetwork()</code>	~50	Analysis — compute all network statistics
<code>printNetworkAnalysis()</code>	~20	Display — formatted network report
<code>printBufferRecommendation()</code>	~15	Display — 3-tier buffer advice

<code>runTests()</code>	~50	Testing — 12 automated test cases
<code>runPresetScenario()</code>	~50	Demo — 4 real-world streaming scenarios
<code>runCustomInput()</code>	~30	Interactive — user enters own data
<code>main()</code>	~40	Entry point — menu loop

9. Sample Output

9.1 Preset Scenario: Mobile 4G Streaming

Scenario: Mobile 4G — 720p Streaming
Duration: 20 seconds | Bitrate: 300 KB/s

Binary Search Trace:
Initial search range: [0, 6000]

```
Step 1: buffer = 3000 KB → SMOOTH → try smaller
Step 2: buffer = 1499 KB → SMOOTH → try smaller
Step 3: buffer = 749 KB → SMOOTH → try smaller
Step 4: buffer = 374 KB → STALL → need more
Step 5: buffer = 561 KB → STALL → need more
Step 6: buffer = 655 KB → SMOOTH → try smaller
Step 7: buffer = 608 KB → STALL → need more
Step 8: buffer = 631 KB → STALL → need more
Step 9: buffer = 643 KB → STALL → need more
Step 10: buffer = 649 KB → STALL → need more
Step 11: buffer = 652 KB → SMOOTH → try smaller
Step 12: buffer = 650 KB → SMOOTH → try smaller
```

```
Minimum Buffer (Binary Search) : 650 KB
Minimum Buffer (Optimal)       : 650 KB
Methods agree                  : YES
```

9.2 Network Analysis Output

Network Condition Analysis

```
-----
Average Bandwidth : 267.5 KB/s (BELOW bitrate!)
Bandwidth Range   : 100 - 400 KB/s
Network Jitter    : 94.1 KB/s (HIGH — unstable!)
Worst 3-sec Window : seconds 5-7 (total: 650 KB)
Stability Index   : 26.3 / 100 [POOR]
-----
```

Buffer Recommendations

```
-----
Minimum (exact)    : 650 KB — zero margin, any extra dip causes stall
Safe (+20% margin) : 825 KB — handles minor fluctuations
Robust (+50% margin): 1089 KB — handles major jitter and spikes
-----
```

9.3 Playback Simulation (partial)

```
Playback Simulation
Initial Buffer: 650 KB | Bitrate: 300 KB/s

-----
Sec | BW(KB/s) | Net Delta | Buffer(KB) | Status
-----
  1 |    350 |    +50 |    700 | filling #####
  2 |    200 |   -100 |    600 | draining #####
  3 |    400 |   +100 |    700 | filling #####
  ...
 17 |    120 |  -180 |     10 | draining #
 18 |    390 |   +90 |    100 | filling #####
 19 |    300 |    +0 |    100 | steady #####
 20 |    200 |  -100 |     0 | draining
-----

Final Buffer: 0 KB | Peak: 700 KB | Stalls: 0
```

10. Test Results

#	TEST DESCRIPTION	EXPECTED	GOT	STATUS
1	Single second, exactly at bitrate	0 KB	0 KB	PASS
2	Single second, below bitrate	50 KB	50 KB	PASS
3	Single second, above bitrate	0 KB	0 KB	PASS
4	Perfectly stable connection	0 KB	0 KB	PASS
5	Always surplus — no buffer needed	0 KB	0 KB	PASS
6	Always deficit — need full pre-load	210 KB	210 KB	PASS
7	Early dip, late recovery	100 KB	100 KB	PASS
8	Deep early deficit, strong recovery	240 KB	240 KB	PASS
9	Alternating low/high bandwidth	50 KB	50 KB	PASS
10	Gradually worsening network	0 KB	0 KB	PASS
11	Variable realistic scenario	N/A	50 KB	PASS
12	Complete network outage	500 KB	500 KB	PASS

✔ All Tests Passed

12 / 12 tests passed. Both algorithms (Binary Search and Closed-Form) agree on every test case.

11. Complexity Analysis

ALGORITHM	TIME COMPLEXITY	SPACE COMPLEXITY	EXPLANATION
Binary Search + Simulation	$O(n \cdot \log(n \cdot B))$	$O(1)$	Binary search over range $[0, n \cdot B]$ does $\log(n \cdot B)$

			iterations. Each iteration runs an $O(n)$ simulation.
Closed-Form Optimal	$O(n)$	$O(1)$	Single pass through the bandwidth array. Tracks running sum and minimum — just 2 variables.
Network Analysis	$O(n)$	$O(1)$	Statistics + sliding window in a combined linear scan.
Playback Simulation	$O(n)$	$O(1)$	Display-only — iterates once through bandwidth array.

Where **n** = number of seconds (stream duration) and **B** = video bitrate (KB/s).

Why Two Algorithms?

The **Binary Search** approach is what the case study requires — it directly demonstrates the DSA concept and is easier to explain step-by-step. The **Closed-Form** approach is mathematically elegant and faster ($O(n)$ vs $O(n \cdot \log(n \cdot B))$), serving as a verification and showing deeper algorithmic thinking. Both produce identical results for every input.

12. Conclusion

Case Study Objectives — All Met

REQUIREMENT	STATUS	IMPLEMENTATION
Determine minimum buffer size	DONE	Both Binary Search and Closed-Form approaches
Binary Search on buffer size	DONE	<code>findMinBufferBinarySearch()</code> with step-by-step trace
Linear simulation to check playback	DONE	<code>canPlaySmoothly()</code> — $O(n)$ feasibility check

Handle network bandwidth variability	DONE	Jitter analysis, sliding window for worst periods, stability index
Optimal buffer size output	DONE	Exact minimum + safe/robust recommendations
Improved user experience	DONE	Interactive menu, preset scenarios, visual simulation

Key Takeaways

- **Binary Search on Answer Space** is a powerful technique that applies whenever the feasibility function is monotonic.
- **Prefix Sums** can sometimes replace binary search entirely, reducing complexity from $O(n \cdot \log(n))$ to $O(n)$.
- **Sliding Window** efficiently finds worst-case consecutive periods in linear time.
- Real-world buffer sizing adds **safety margins** scaled by network stability — the mathematical minimum is a starting point, not a deployment value.

DSA Mini Project — Case Study 5: Video Streaming Buffer Optimization
Built with C++17 | Compiled with g++ -std=c++17 -O2 -Wall -Wextra | All 12 tests passed